



If you know Playwright, you already know mobile testing

Open-source · Playwright test runner · Appium 3 underneath

Syam Sasi · Taqelah



Disclaimer

This is not a product pitch.

Today I'm sharing **only the open-source part** of taqwright —
free to use, absolutely.

No sign-up · no account · no paid tier needed for anything you see today.



Mobile test automation today 🙄

Raw Appium / WebDriverIO is powerful — but:

- ⏰ **Explicit waits everywhere** → `waitForDisplayed({ timeout: 20000 })`
- 🧱 **Session boilerplate** → capabilities, server start, teardown
- 🎯 **Brittle selectors** → `UiSelector().className(...).instance(0)`
- 🐢 **Flaky runs** → hardcoded sleeps "fix" it until they don't

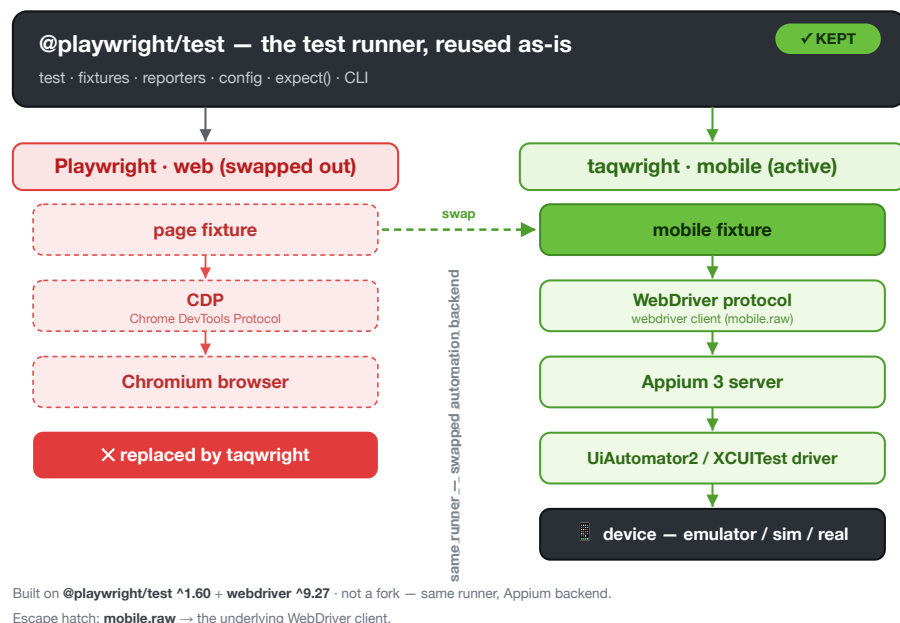
Meanwhile on web, Playwright made all of this **go away**.

Why can't mobile feel like that?



What taqwright is

The Playwright test runner + a flat **mobile** locator API, on top of Appium 3.



Not a fork. `@playwright/test` is reused as-is — runner, fixtures, `expect`, HTML report, retries, workers, sharding, projects. Only the backend is swapped: `page` → **mobile** → WebDriver → Appium 3 → UiAutomator2 / XCUITest.



Part 1 · Install

From zero to a runnable project



Install @taqwright/taqwright

```
nvm install 26 && nvm use 26 # ⚠ Node 24+ required  
npx @taqwright/taqwright init demo # scaffold + install into ./demo  
cd demo
```

Package: `@taqwright/taqwright` · command: `taqwright` — inside the project it's just `npx taqwright ...`.

Existing project? `npm i -D @taqwright/taqwright`

You need

- **Node 24+** — the only hard fail
- Android: JDK · Android SDK · Appium 3
- iOS: macOS + **Xcode** (manual)

Don't have the Android bits?

`init` **probes your machine** and offers to install JDK + SDK + Appium (~700 MB) + an emulator (~1 GB).



taqwright init — the interactive flow

1. Test folder name (*tests*)
2. Platform [android/ios/both] → **android**
3. Android → **probes the toolchain** → auto-install if missing
4. Also create an emulator? (*or pick an existing AVD*)
5. **Download the demo app?** → yes
6. Run npm install? → yes

Android toolchain:

- ✓ JDK
- ✓ Android SDK (adb)
- ✓ Appium 3.x
- ✓ uiautomator2 driver
- ✓ Android emulator (AVD)
- detected a working Android toolchain – skipping install.

Created:

```
package.json    taqwright.config.ts
tsconfig.json   tests/example.spec.ts
.npmrc          .gitignore
app/DemoApp-v1.0.0.apk
```

Non-interactive / Cl: `npx @taqwright/taqwright init demo --platform android --demo-app --yes`



Before the first run — two health checks 🩺

```
npx taqwright doctor      # is my machine ready? (13 checks)
npx taqwright devices     # what can I target?
```

```
taqwright doctor (v1.0.0)
[ok] Node.js 24+           - v26.4.0
[ok] adb (Android SDK)     - on PATH
[ok] Appium (test server)  - v3.3.1
[ok] Appium drivers        - uiautomator2, xcuitest
...
```

Only **Node 24** is a hard fail — everything else is a warning, and you only need the rows for **your** platform. Appium **auto-starts** — no second terminal.



Part 2 · Run the 3 ready-made tests

What `init` gives you, out of the box



tests/example.spec.ts — scaffolded for you

```
import { test, expect } from '@taqwright/taqwright';

test('user can log in to the demo app', async ({ mobile }) => {
  await mobile.getByXPath("//*[@hint='Username']").fill('emma@demoapp.com');
  await mobile.getByXPath("//*[@hint='Password']").fill('10203040');
  await mobile.getByUiSelector('new UiSelector().description("Login")').click();
  await expect(mobile.getByUiSelector('new UiSelector().description("View All")')).toBeVisible();
});

test('login fails with invalid username & password', async ({ mobile }) => { /* ... */
  await expect(mobile.getByXPath("//*[contains(@content-desc, 'Invalid username or password.')]")).toBeVisible();
});

test('login is blocked without username & password', async ({ mobile }) => { /* ... */
  await expect(mobile.getByUiSelector('new UiSelector().description("Please enter your username")')).toBeVisible();
});
```

One **happy path**, two **negative paths** — against the demo app `init` downloaded.



taqwright.config.ts — already wired up by `init`

```
projects: [{
  name: 'android',
  use: {
    platform: Platform.ANDROID,
    device: {
      provider: 'emulator',
      name: 'taqwright_api34',      // ← your AVD
    },
    appium: {
      autoStart: true,             // ← Appium on :4723
      autoStartDevice: true,       // ← boots that AVD
    },
    resetBetweenTests: true,       // ← fresh app each test
    buildPath: './app/DemoApp-v1.0.0.apk',
    appBundleId: 'com.taqelah.demo_app',
  },
}]
```

Emulator — auto-selected

`init` lists the AVDs on your machine:

- **one** → used automatically
- **several** → you pick a number
- **none** → offers to create `taqwright_api34`

APK — auto-downloaded

"Download the demo app? → yes" pulls `DemoApp-v1.0.0.apk` into `app/` and fills in `buildPath` + `appBundleId`.

Your own app? Swap those two lines.



Run them

```
npx taqwright test --project android # android only – runs all 3
npx taqwright show-report           # HTML report, same as Playwright
```

What happens for you

- 📱 emulator **booted** if it's off
- 🔌 Appium **auto-started** on :4723
- 📦 app **reinstalled + relaunched** before every test → clean state
- 🗑️ session torn down after

What you get back

- ✅ `list` output in the terminal
- 📊 HTML report
- 🔍 **trace** on failure — timeline + a screenshot **after every action**



Video + trace — same knobs as Playwright

```
projects: [{
  name: 'android',
  use: {
    // ...platform, device, app...
    trace: 'on-failure', // ← trace
    video: 'on-failure', // ← video
    // network: 'on-failure',
  },
}]
```

Values: 'off' · 'on' · 'on-failure' · 'retain-on-failure'

network = HAR capture of app traffic

	Playwright	taqwright
Where	use: { ... }	use: { ... } — same
Trace	.zip → trace viewer	self-contained trace.html
Video	.webm	device screen.mp4
View	show-report	show-report — same

Trace: clickable timeline · screenshot **after every action** · page-source XML

💡 Cl: use 'on-failure' — trace adds ~100–300 ms per action.



Why no `sleep()` anywhere? Auto-waiting

✗ One-shot, racy

```
await mobile.waitForTimeout(2000);  
if (await loc.isVisible()) {  
  // checks ONCE, right now  
}
```

✓ Auto-retrying

```
await expect(loc).toBeVisible();  
// polls every 200 ms  
// until visible or 30 s
```

- **Actions** (`click`, `fill`, `swipe` ...) wait for **visible + enabled**
- **Assertions** retry until they pass or time out
- Hardcoded sleeps are the **#1 cause of flake** — and unnecessary here



Part 3 · Run in parallel ⚡

More emulators, faster runs — one config change



How parallel runs work

Default: serial — `workers: 1`, one emulator, one test at a time.

Parallel: `workers: N` →

- **N tests at once**
- each worker gets **its own emulator**
- ...and **its own Appium**: worker 0 → `:4723`, worker 1 → `:4724` (automatic)

Two rules

1. `workers ≤ number of devices`
(too few → fails fast at start-up)
2. `fullyParallel: true` → tests **inside one file** spread across workers too

Our 3 ready-made tests live in **one file** — without `fullyParallel` they'd all go to one worker.



Add a parallel project — existing one untouched

```
export default defineConfig({
  fullyParallel: true,           // ← 1
  projects: [
    { name: 'android', /* ... unchanged ... */ },
    {
      name: 'android-parallel',   // ← new project
      workers: 2,                // ← 2
      use: {
        platform: Platform.ANDROID,
        device: {
          provider: 'emulator',
          pool: [                 // ← 3
            { name: 'taqwright_api34', udid: 'emulator-5554' },
            { name: 'Pixel_7_API_34', udid: 'emulator-5556' },
          ],
        },
        appium: { autoStart: true, autoStartDevice: true },
        // buildPath / appBundleId / resetBetweenTests — as android
      },
      testMatch: ['**/android/**'],
    },
  ],
});
```

New project — keep `android` for serial runs, pick parallel with `--project`

① **fullyParallel** — split tests within a file (top-level only)

② **workers: 2** — 2 tests at once

③ **pool** — one emulator per worker: AVD **name** (to boot it) + **udid** (its serial)
→ from `npx taqwright devices`

No names to copy? → let taqwright find the emulators (next slide)



Or let taqwright find the emulators

```
{
  name: 'android-auto',
  workers: 2,
  use: {
    platform: Platform.ANDROID,
    device: {
      provider: 'emulator',
      autoDiscover: true,      // ← that's it
    },
    appium: { autoStart: true },
    // app settings – same as android
  },
  testMatch: ['**/android/**'],
}
```

pool — you list each emulator

- ✓ exact devices, boots them for you
- ✗ AVD names + serials are **machine-specific**

autoDiscover — taqwright finds them

- ✓ **no names** — same config on every laptop & CI
- ✓ each worker gets its own emulator
- ⚠ **boot the emulators first** — 2 workers → 2

running



Run it ⚡

```
npx taqwright devices
npx taqwright test --project android-parallel
npx taqwright test --project android-auto
npx taqwright test --project android
npx taqwright show-report
```

need 2 AVDs
pool: 3 tests, 2 emulators, at once
autoDiscover: same, no names
same tests, serial – compare the time

What you'll see

- 📱 📱 **two emulators** driven at once
- 🔌 Appium on **:4723** and **:4724**
- ⌚ wall-clock time drops — 3 tests finish in ~2 test-lengths

Scale further

- more emulators → bump **workers** + add to **pool**
- cloud: **workers: N** on LambdaTest / BrowserStack — **no pool**, the grid supplies devices



Part 4 · Record with codegen

Don't hand-write locators — record them



Codegen — Playwright-style recording for mobile

```
npx taqwright codegen          # opens the inspector at localhost:4280
```

One screen, four panels

-  **live screen mirror** — tap / type / swipe the real device in your browser
-  **view tree** — the live accessibility tree
-  **locators** — every candidate, **ranked** + **uniqueness-checked**
-  **recorded script** — taqwright code, written as you act

Smart locators

- prefers **stable ids** over positions
- checks each one is **unique** on screen
-  flags **fragile** (positional / xpath) picks
- click **assert** → a retrying `expect ( )`

Reads `taqwright.config.ts` — same device, same app, Appium auto-starts. Works on cloud devices too.



Live — record the add-to-cart flow

```
npx taqwright codegen
```

1. Log in — `emma@demoapp.com` / `10203040`
2. **View All** → open **Black Sequin Mini**
3. **Add to Cart** → **VIEW CART**
4. Select **Proceed to Checkout** → **assert visible**
5. Copy the **Recorded script** → `tests/add-to-cart.spec.ts`

```
npx taqwright test tests/add-to-cart.spec.ts --project android
```



What codegen wrote 🖋️

```
test("recorded test", async ({ mobile }) => {  
  await mobile.getByXpath("//*[@hint='Username']").fill("emma@demoapp.com");  
  await mobile.getByXpath("//*[@hint='Password']").fill("10203040");  
  await mobile.getByUiSelector("new UiSelector().description(\"Login\")").click();  
  await mobile.getByUiSelector("new UiSelector().description(\"View All\")").click();  
  await mobile.getByUiSelector("new UiSelector().descriptionContains(\"Black Sequin Mini\")").click();  
  await mobile.getByUiSelector("new UiSelector().description(\"Add to Cart\")").click();  
  await mobile.getByUiSelector("new UiSelector().description(\"VIEW CART\")").click();  
  await expect(mobile.getByUiSelector("new UiSelector().description(\"Proceed to Checkout\")")).toBeVisible();  
});
```

✅ Runs green on Android · ❌ every locator is **Android-only**. Next: the same flow on iOS.



Same flow — recorded on iOS 🍏

```
import { test, expect } from '@taqwright/taqwright';

test("recorded test", async ({ mobile }) => {
  await mobile.getById("Username").fill("emma@demoapp.com");
  await mobile.getById("Password").fill("10203040");
  await mobile.getById("Login").click();
  await mobile.getById("View All").click();
  await mobile.getById("Black Sequin Mini\n$119.99").click();
  await mobile.getById("Add to Cart").click();
  await mobile.getById("VIEW CART").click();
  await expect(mobile.getById("Proceed to Checkout")).toBeVisible();
});
```

Same 8 steps · iOS exposes **accessibility ids** → every locator is a clean `getById` · but **not one** matches the Android script.



Run it on iOS — two changes to the config 🍏

```
{
  name: 'ios',
  use: {
    platform: Platform.IOS,
    device: {
      provider: 'emulator',
      name: 'iPhone 17',           // ← 1
      osVersion: '26.5',         // ← 1
      udid: '30FA4507-...-A40DA229CC13', // ← 1
    },
    resetBetweenTests: true,      // ← 2
    buildPath: './app/DemoApp-v1.0.0.app', // ← 2
    appBundleId: 'com.taqelah.demoApp', // ← 2
  },
  testMatch: ['**/ios/**'],
}
```

① Pick the simulator

`init` wrote `name: /iPhone/`. Pin yours from:

```
npx taqwright devices
```

→ copy `name · osVersion · udid`

② Point at the iOS app

`init` only downloads the Android APK → add the simulator build (`.app`) + its bundle id.

Run only the new test

```
npx taqwright test --project ios \
  -g "Add to Cart"
```



Same tests, real devices — LambdaTest

```
{
  name: 'lambdatest-android',
  use: {
    platform: Platform.ANDROID,
    device: { provider: 'lambdatest', name: 'Pixel 8', osVersion: '14' },
    buildPath: 'lt://APP101602071790272933857933', // uploaded APK
    appBundleId: 'com.taqelah.demo_app',
  },
  testMatch: ['**/android/**'],
},
{
  name: 'lambdatest-ios',
  use: {
    platform: Platform.IOS,
    device: { provider: 'lambdatest', name: 'iPhone 15', osVersion: '17' },
    buildPath: 'lt://APP10160372241789400782563395', // uploaded iOS build
    appBundleId: 'com.taqelah.demoApp',
  },
  testMatch: ['**/ios/**'],
},
}
```

Credentials — env, not config

```
export LAMBDATEST_USERNAME=...
export LAMBDATEST_ACCESS_KEY=...
```

Run

```
npx taqwright test \
  --project lambdatest-android
npx taqwright test \
  --project lambdatest-ios
```

- **Same test files** — only the project config changes
- No local Appium / emulator
- 📹 video + device logs on the LambdaTest dashboard



Codegen records gestures too 🖐️

Select an element → Actions tab → pick the gesture. Screen-wide swipes & scrolls come from the screen controls.

Gesture	What codegen writes
👉 Swipe left / right	<code>await mobile.swipe('left') · element: await locator.swipeLeft() / .swipeRight()</code>
👆 Scroll up / down	<code>await mobile.scroll('down') · to an element: await locator.scrollIntoView()</code>
🖐️ Drag & drop	<code>await source.dragTo(target)</code>
👉 Long press	<code>await locator.longPress()</code>
👉👉 Double tap	<code>await locator.doubleTap()</code>
🔍 Zoom in / out	<code>await locator.pinchOut()</code> → zoom in · <code>await locator.pinchIn()</code> → zoom out

Same auto-waiting as `click` — every element gesture waits for **visible + enabled** first. Works on Android **and** iOS.



Try it tonight

```
npx @taqwright/taqwright init demo # your project + 3 ready-made tests
cd demo && npx taqwright test

git clone https://github.com/taqelah/taqwright-tutorial # or: the full tutorial
cd taqwright-tutorial/tutorial_11_page_object_lab && npm install
npm run test:android
```

- 📱 **12 hands-on labs:** codegen · config · parallel · sharding · retries · traces · page objects · cloud
- 📦 Demo app committed in the repo — nothing to download

★ Star it · 🐛 issues, ideas, PRs welcome → github.com/taqelah/taqwright



 **Thank you! Questions?**

github.com/taqelah/taqwright

 linkedin.com/in/syam-sasi ·  taqelah.sg